

Milestone 1 — NumPy Solutions

Both complete NumPy problem sets from the supplied PDF.

Question 1 — City Air Quality (AQI)

5 functions: create array, validate, statistics, categorize, unhealthy streak.

```
import numpy as np
```

```
def create_aqi_array(values: list) -> np.ndarray:
    return np.array(values, dtype=np.int64)

def validate_aqi_array(arr: np.ndarray) -> bool:
    if arr.size == 0:
        return False
    return np.all((arr >= 0) & (arr <= 100))

def compute_aqi_stats(arr: np.ndarray) -> tuple:
    average = round(np.mean(arr), 2)
    std = round(np.std(arr), 2)
    maximum = round(np.max(arr), 2)
    minimum = round(np.min(arr), 2)
    return (average, std, maximum, minimum)

def categorize_aqi(arr: np.ndarray) -> np.ndarray:
    return np.where(arr <= 50, "Good",
                    np.where(arr <= 100, "Moderate",
                              np.where(arr <= 150, "USG",
                                        np.where(arr <= 200, "Unhealthy",
                                                  np.where(arr <= 300, "Very Unhealthy", "Hazardous"))))))

def longest_unhealthy_streak(arr: np.ndarray) -> int:
    unhealthy = arr >= 151
    max_streak = 0
    current_streak = 0
    for value in unhealthy:
        if value:
            current_streak += 1
            if current_streak > max_streak:
                max_streak = current_streak
        else:
            current_streak = 0
    return max_streak
```

Key outputs: (161.0, 103.99, 325, 52) and longest unhealthy streak = 2.

Question 2 — Sensor Reading Analysis System

6 functions: create array, validate, statistics, filter, label, format.

```
import numpy as np
```

```
class SensorAnalyzer:
```

```
    def create_sensor_array(self, sensor_values: list) -> np.ndarray:
        return np.array(sensor_values, dtype=float)

    def validate_sensor_array(self, sensor_array: np.ndarray) -> bool:
        if sensor_array.size == 0:
            return False
        return np.all(sensor_array > 0)

    def compute_sensor_statistics(self, sensor_array: np.ndarray) -> tuple:
        total = np.sum(sensor_array)
        average = round(np.mean(sensor_array), 1)
        maximum = np.max(sensor_array)
        return (total, average, maximum)

    def filter_extreme_readings(self, sensor_array: np.ndarray) -> np.ndarray:
        result = sensor_array.astype(float).copy()
        result[result >= 50.0] *= 0.90
        return result

    def label_high_sensors(self, sensor_array: np.ndarray) -> np.ndarray:
        mean_value = np.mean(sensor_array)
        return np.where(sensor_array > mean_value, "High", "Normal")

    def format_sensor_readings(self, sensor_array: np.ndarray) -> np.ndarray:
        return np.array([f"{x:.2f} units" for x in sensor_array])
```

For [100, 50, 60], the written PDF rule gives [90, 45, 54] because values ≥ 50 are reduced by 10%.

NumPy Quick Revision

`np.array()\nnp.mean()\nnp.std()\nnp.max()\nnp.min()\nnp.sum()\nnp.all()\nnp.where()\narr[arr >= 50] = arr[arr >= 50]`